

P3731R0

#embed Preprocessor Parameter Order

Published Proposal, 2025-06-05

Authors:

[JeanHeyd Meneide](#)

[Shepherd \(Shepherd's Oasis LLC\)](#)

Latest:

https://thephd.dev/_vendor/future_cxx/papers/d1967.html

Paper Source:

[GitHub ThePhD/future_cxx](#)

Implementation:

[GitHub ThePhD/embed](#)

Project:

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Audience:

EWG, SG22

Abstract

Preprocessor embed has a fixed behavior no matter which parameter ordering is chosen. This paper attempts to enforce or encourage the ordering of preprocessor parameters to prevent mental mistakes from users.

Table of Contents

1 Changelog

1.1 Revision 0 - June 5th, 2025

2 Introduction & Motivation

3 Design

4 Wording

4.1 Intent

4.2 Proposed Language Wording

4.2.1 **HARD ERROR VERSION:** Add a new paragraph after (currently) paragraph 12 in [cpp.embed.gen]

4.2.2 **RECOMMENDED PRACTICE:** Add a new paragraph after (currently) paragraph 12 in [cpp.embed.gen]

5 Acknowledgements

References

Informative References

§ 1. Changelog

§ 1.1. Revision 0 - June 5th, 2025

— Initial release.

§ 2. Introduction & Motivation

During the Wednesday, June 4th, 2025 [SG22 meeting](#), a few individuals voiced some potential concern that the ordering of parameters -- particularly `limit` and `offset` -- could confuse users. While it was pointed out that the formula that applied was the same no matter the ordering of parameters and that the entire `#embed` directive needed to be processed no matter what order was used, some still voiced a preference for providing a hard error about providing e.g. the `offset(...)` parameter in an `#embed` directive after the `limit(...)` parameter.

To answer their concerns, this paper proposes 2 versions of the same text. One is a hard-error based wording (ill-formed "shall" in C++, a shall-based wording under a Constraints section in C++). The other is a soft-error/warning based wording ("Recommended practice" section in both C++ and C). It is left up to the Committee to choose.

As it stands now, no implementation currently imposes an ordering. The parameters can be provided in any order and it does not change behavior. This is both the C23 behavior for existing parameters, and the extension parameters such as `gnu::offset` and `clang::offset`.

NOTE: The paper authors for `#embed` and the `offset` parameter ([p3540], [n3490]), as well as this paper, do not care and thinks this is not worth the investment. Nevertheless, the Committees can decide if they want it, and in what form.

§ 3. Design

The design for this is not any kind of in-depth and is simple. According to both the discussion in SG22 and just generally understanding the consequences of justifying "the order of operations is important" (even if it, again, does not matter for existing implementations and nobody has implemented it like that), there's 3 general principles here.

- `offset` needs to appear before `limit` because it is applied in the algorithm before the `limit` is calculated.
- `limit` needs to appear before `if_empty`, `prefix`, and `suffix`.
- if `limit` is not present, `offset` still needs to come before `if_empty`, `prefix`, and `suffix`.

The reason for the second and third constraints is because needs to be in this order because it changes whether or not the resource is considered empty or not (and thus affects whether or not any of the following parameters trigger).

§ 4. Wording

This wording is relative to C++'s latest working draft.

§ 4.1. Intent

The intent of this wording is to provide 2 options for the Committee to chose from:

- ill-formed if the order of parameters is not in the way that the Committee demands it;
- or, recommended practice to diagnose the ordering of parameters.

The choice is left to a vote by the Evolution Working Groups.

§ 4.2. Proposed Language Wording

§ 4.2.1. HARD ERROR VERSION: Add a new paragraph after (currently) paragraph 12 in [cpp.embed.gen]

...

- 11 The method by which a sequence of preprocessing tokens between a < and a > preprocessing token pair or a pair of " characters is combined into a single resource name preprocessing token is implementation-defined.

1[†] ✨ Certain preprocessor parameters have to appear before other preprocessor embed parameters in a specific order.

- offset, if present, shall appear before limit, if present.
- both offset and limit, if present, shall appear before if_empty.prefix, and suffix.

[Note: Similar diagnostics for implementation-defined embed parameters, which may be better understood in a specific order, also have the potential to clarify semantics. - end Note]

§ 4.2.2. RECOMMENDED PRACTICE: Add a new paragraph after (currently) paragraph 12 in [cpp.embed.gen]

...

- 11 The method by which a sequence of preprocessing tokens between a < and a > preprocessing token pair or a pair of " characters is combined into a single resource name preprocessing token is implementation-defined.

1✦ Recommended Practice: Certain embed parameters should appear before other pre-processor embed parameters in a specific order.

- offset, if present, should appear before limit, if present.
- both offset and limit, if present, should appear before any of if_empty, prefix, or suffix.

This aids in comprehension, as no matter where an offset appears, it is always applied before the value in a limit parameter. Similarly, as limit and offset can change if a resource is considered empty or not, each of these should appear in their proper order before if_empty, suffix, or prefix. Implementations should consider applying similar diagnostics for implementation-defined embed parameters which might potentially be better understood in a specific order.

§ 5. Acknowledgements

Thank you to SG22 for getting the discussion of this paper off the ground.

§ References

§ Informative References

[N3490]

JeanHeyd Meneide; Shepherd. *N3490 - #embed Synchronization*. June 5th, 2025. URL: https://thephd.dev/_vendor/future_cxx/papers/C%20-%20embed%20Synchronization.html

[P3540]

JeanHeyd Meneide; Shepherd. *P3540 - #embed offset parameter*. June 5th, 2025. URL: https://thephd.dev/_vendor/future_cxx/papers/d3540.html

[SG22-MINUTES-0]

Study Group 22: C and C++ Compatibility. *June 4th, 2025 Meeting Minutes*. June 4th, 2025.

URL: <https://github.com/sg22-c-cpp-standard-compatibility/sg-compatibility/blob/main/README.md#june-4th-2025>